real axis. A root-finding method such as bisection or Newton's method can then be employed to refine the intervals. The corresponding eigenvectors can then be found by inverse iteration (see §11.8).

Procedures based on these ideas can be found in [2,3]. If, however, more than a small fraction of all the eigenvalues and eigenvectors is required, then the factorization method next considered is much more efficient.

11.4.2 The QR and QL Algorithms

The basic idea behind the QR algorithm is that any real matrix can be decomposed in the form

$$\mathbf{A} = \mathbf{Q} \cdot \mathbf{R} \tag{11.4.1}$$

where $\mathbf{Q}$ is orthogonal and $\mathbf{R}$ is upper triangular. For a general matrix, the decomposition is constructed by applying Householder transformations to annihilate successive columns of $\mathbf{A}$ below the diagonal (see §2.10).

Now consider the matrix formed by writing the factors in (11.4.1) in the opposite order:

$$\mathbf{A}' = \mathbf{R} \cdot \mathbf{Q} \tag{11.4.2}$$

Since $\mathbf{Q}$ is orthogonal, equation (11.4.1) gives $\mathbf{R} = \mathbf{Q}^T \cdot \mathbf{A}$. Thus equation (11.4.2) becomes

$$\mathbf{A}' = \mathbf{Q}^T \cdot \mathbf{A} \cdot \mathbf{Q} \tag{11.4.3}$$

We see that $\mathbf{A}'$ is an orthogonal transformation of $\mathbf{A}$.

You can verify that a QR transformation preserves the following properties of a matrix: symmetry, tridiagonal form, and Hessenberg form (to be defined in §11.6).

There is nothing special about choosing one of the factors of $\mathbf{A}$ to be upper triangular; one could equally well make it lower triangular. This is called the QL algorithm, since

$$\mathbf{A} = \mathbf{Q} \cdot \mathbf{L} \tag{11.4.4}$$

where $\mathbf{L}$ is lower triangular. (The standard, but confusing, nomenclature R and L stands for whether the *right* or *left* of the matrix is nonzero.)

Recall that in the Householder reduction to tridiagonal form in §11.3, we started in column $n-1$ of the original matrix. To minimize roundoff, we then exhorted you to put the biggest elements of the matrix in the lower right-hand corner, if you can. If we now wish to diagonalize the resulting tridiagonal matrix, the QL algorithm will have smaller roundoff than the QR algorithm, so we shall use QL henceforth.

The QL algorithm consists of a *sequence* of orthogonal transformations:

$$\begin{aligned} \mathbf{A}_s &= \mathbf{Q}_s \cdot \mathbf{L}_s \\ \mathbf{A}_{s+1} &= \mathbf{L}_s \cdot \mathbf{Q}_s \qquad (= \mathbf{Q}_s^T \cdot \mathbf{A}_s \cdot \mathbf{Q}_s) \end{aligned} \tag{11.4.5}$$

The following (nonobvious!) theorem is the basis of the algorithm for a general matrix $\mathbf{A}$: (i) If $\mathbf{A}$ has eigenvalues of different absolute value $|\lambda_i|$, then $\mathbf{A}_s \to$ [lower triangular form] as $s \to \infty$. The eigenvalues appear on the diagonal in increasing order of absolute magnitude. (ii) If $\mathbf{A}$ has an eigenvalue $|\lambda_i|$ of multiplicity p, $\mathbf{A}_s \to$ [lower triangular form] as $s \to \infty$, except for a diagonal block matrix of order p, whose eigenvalues $\to \lambda_i$. The proof of this theorem is fairly lengthy; see, for example, [4].

The workload in the QL algorithm is $O(n^3)$ per iteration for a general matrix, which is prohibitive. However, the workload is only $O(n)$ per iteration for a tridiagonal matrix and $O(n^2)$ for a Hessenberg matrix, which makes it highly efficient on these forms.

In this section we are concerned only with the case where $\mathbf{A}$ is a real, symmetric, tridiagonal matrix. All the eigenvalues λ_i are thus real. According to the theorem, if any λ_i has a multiplicity p, then there must be at least $p-1$ zeros on the sub- and superdiagonals. Thus the matrix can be split into submatrices that can be diagonalized separately, and the complication of diagonal blocks that can arise in the general case is irrelevant.

In the proof of the theorem quoted above, one finds that in general a superdiagonal element converges to zero like

$$a_{ij}^{(s)} \sim \left(\frac{\lambda_i}{\lambda_j}\right)^s \tag{11.4.6}$$

Although $\lambda_i < \lambda_j$, convergence can be slow if λ_i is close to λ_j. Convergence can be accelerated by the technique of *shifting*: If k is any constant, then $\mathbf{A} - k\mathbf{1}$ has eigenvalues $\lambda_i - k$. If we decompose

$$\mathbf{A}_s - k_s\mathbf{1} = \mathbf{Q}_s \cdot \mathbf{L}_s \tag{11.4.7}$$

so that

$$\begin{aligned} \mathbf{A}_{s+1} &= \mathbf{L}_s \cdot \mathbf{Q}_s + k_s\mathbf{1} \\ &= \mathbf{Q}_s^T \cdot \mathbf{A}_s \cdot \mathbf{Q}_s \end{aligned} \tag{11.4.8}$$

then the convergence is determined by the ratio

$$\frac{\lambda_i - k_s}{\lambda_j - k_s} \tag{11.4.9}$$

The idea is to choose the shift k_s at each stage to maximize the rate of convergence. A good choice for the shift initially would be k_s close to λ_0, the smallest eigenvalue. Then the first row of off-diagonal elements would tend rapidly to zero. However, λ_0 is not usually known a priori. A very effective strategy in practice (although there is no proof that it is optimal) is to compute the eigenvalues of the leading 2×2 diagonal submatrix of $\mathbf{A}$. Then set k_s equal to the eigenvalue closer to a_{00}.

More generally, suppose you have already found r eigenvalues of $\mathbf{A}$. Then you can *deflate* the matrix by crossing out the first r rows and columns, leaving

$$\mathbf{A} = \begin{bmatrix} 0 & & & \cdots & \cdots & & & 0 \\ & \cdots & & & & & & \\ & & 0 & & & & & \\ \vdots & & & d_r & e_r & & & \vdots \\ \vdots & & & e_r & d_{r+1} & & & \\ & & & & & \cdots & & 0 \\ & & & & & & d_{n-2} & e_{n-2} \\ 0 & & & \cdots & & 0 & e_{n-2} & d_{n-1} \end{bmatrix} \tag{11.4.10}$$

Choose k_s equal to the eigenvalue of the leading 2×2 submatrix that is closer to d_r. One can show that the convergence of the algorithm with this strategy is generally cubic (and at worst quadratic for degenerate eigenvalues). This rapid convergence is what makes the algorithm so attractive.

Note that with shifting, the eigenvalues no longer necessarily appear on the diagonal in order of increasing absolute magnitude. The routine `eigsrt` (§11.1) can be used if required.

As we mentioned earlier, the QL decomposition of a general matrix is effected by a sequence of Householder transformations. For a tridiagonal matrix, however, it is more efficient to use plane rotations $\mathbf{P}_{pq}$. One uses the sequence $\mathbf{P}_{01}, \mathbf{P}_{12}, \ldots, \mathbf{P}_{n-2,n-1}$ to annihilate the elements $a_{01}, a_{12}, \ldots, a_{n-2,n-1}$. By symmetry, the subdiagonal elements $a_{10}, a_{21}, \ldots, a_{n-1,n-2}$ will be annihilated too. Thus each $\mathbf{Q}_s$ is a product of plane rotations:

$$\mathbf{Q}_s^T = \mathbf{P}_1^{(s)} \cdot \mathbf{P}_2^{(s)} \cdots \mathbf{P}_{n-1}^{(s)} \tag{11.4.11}$$

where $\mathbf{P}_i$ annihilates $a_{i-1,i}$. Note that it is $\mathbf{Q}^T$ in equation (11.4.11), not $\mathbf{Q}$, because we defined $\mathbf{L} = \mathbf{Q}^T \cdot \mathbf{A}$.

11.4.3 QL Algorithm with Implicit Shifts

The algorithm as described so far can be very successful. However, when the elements of $\mathbf{A}$ differ widely in order of magnitude, subtracting a large k_s from the diagonal elements can lead to loss of accuracy for the small eigenvalues. This difficulty is avoided by the QL algorithm with *implicit shifts*. The implicit QL algorithm is mathematically equivalent to the original QL algorithm, but the computation does not require $k_s\mathbf{1}$ to be actually subtracted from $\mathbf{A}$.

The algorithm is based on the following lemma: If $\mathbf{A}$ is a symmetric nonsingular matrix and $\mathbf{B} = \mathbf{Q}^T \cdot \mathbf{A} \cdot \mathbf{Q}$, where $\mathbf{Q}$ is orthogonal and $\mathbf{B}$ is tridiagonal with positive off-diagonal elements, then $\mathbf{Q}$ and $\mathbf{B}$ are fully determined when the last row of $\mathbf{Q}^T$ is specified. Proof: Let $\mathbf{q}_i^T$ denote the row vector i of the matrix $\mathbf{Q}^T$. Then $\mathbf{q}_i$ is the column vector i of the matrix $\mathbf{Q}$. The relation $\mathbf{B} \cdot \mathbf{Q}^T = \mathbf{Q}^T \cdot \mathbf{A}$ can be written

$$\begin{bmatrix} \beta_0 & \gamma_0 & & & \\ \alpha_1 & \beta_1 & \gamma_1 & & \\ & & \vdots & & \\ & & \alpha_{n-2} & \beta_{n-2} & \gamma_{n-2} \\ & & & \alpha_{n-1} & \beta_{n-1} \end{bmatrix} \cdot \begin{bmatrix} \mathbf{q}_0^T \\ \mathbf{q}_1^T \\ \vdots \\ \mathbf{q}_{n-2}^T \\ \mathbf{q}_{n-1}^T \end{bmatrix} = \begin{bmatrix} \mathbf{q}_0^T \\ \mathbf{q}_1^T \\ \vdots \\ \mathbf{q}_{n-2}^T \\ \mathbf{q}_{n-1}^T \end{bmatrix} \cdot \mathbf{A} \tag{11.4.12}$$

Row $n-1$ of this matrix equation is

$$\alpha_{n-1}\mathbf{q}_{n-2}^T + \beta_{n-1}\mathbf{q}_{n-1}^T = \mathbf{q}_{n-1}^T \cdot \mathbf{A} \tag{11.4.13}$$

Since $\mathbf{Q}$ is orthogonal,

$$\mathbf{q}_{n-1}^T \cdot \mathbf{q}_m = \delta_{n-1,m} \tag{11.4.14}$$

Thus, if we postmultiply equation (11.4.13) by $\mathbf{q}_{n-1}$, we find

$$\beta_{n-1} = \mathbf{q}_{n-1}^T \cdot \mathbf{A} \cdot \mathbf{q}_{n-1} \tag{11.4.15}$$

which is known since $\mathbf{q}_{n-1}$ is known. Then equation (11.4.13) gives

$$\alpha_{n-1}\mathbf{q}_{n-2}^T = \mathbf{z}_{n-2}^T \tag{11.4.16}$$

where

$$\mathbf{z}_{n-2}^T \equiv \mathbf{q}_{n-1}^T \cdot \mathbf{A} - \beta_{n-1}\mathbf{q}_{n-1}^T \tag{11.4.17}$$

is known. Therefore

$$\alpha_{n-1}^2 = \mathbf{z}_{n-2}^T \mathbf{z}_{n-2}, \tag{11.4.18}$$

or

$$\alpha_{n-1} = |\mathbf{z}_{n-2}| \tag{11.4.19}$$

and

$$\mathbf{q}_{n-2}^T = \mathbf{z}_{n-2}^T/\alpha_{n-1} \tag{11.4.20}$$

(where α_{n-1} is nonzero by hypothesis). Similarly, one can show by induction that if we know $\mathbf{q}_{n-1}, \mathbf{q}_{n-2}, \dots, \mathbf{q}_{n-j}$ and the α's, β's, and γ's up to level $n-j$, one can determine the quantities at level $n-(j+1)$.

To apply the lemma in practice, suppose one can somehow find a tridiagonal matrix $\bar{\mathbf{A}}_{s+1}$ such that

$$\bar{\mathbf{A}}_{s+1} = \bar{\mathbf{Q}}_s^T \cdot \bar{\mathbf{A}}_s \cdot \bar{\mathbf{Q}}_s \tag{11.4.21}$$

where $\bar{\mathbf{Q}}_s^T$ is orthogonal and has the same last row as $\mathbf{Q}_s^T$ in the original QL algorithm. Then $\bar{\mathbf{Q}}_s = \mathbf{Q}_s$ and $\bar{\mathbf{A}}_{s+1} = \mathbf{A}_{s+1}$.

Now, in the original algorithm, from equation (11.4.11) we see that the last row of $\mathbf{Q}_s^T$ is the same as the last row of $\mathbf{P}_{n-1}^{(s)}$. But recall that $\mathbf{P}_{n-1}^{(s)}$ is a plane rotation designed to annihilate the $(n-2, n-1)$ element of $\mathbf{A}_s - k_s\mathbf{1}$. A simple calculation using the expression (11.1.1) shows that it has parameters

$$c = \frac{d_{n-1} - k_s}{\sqrt{e_{n-1}^2 + (d_{n-1} - k_s)^2}}, \qquad s = \frac{-e_{n-2}}{\sqrt{e_{n-1}^2 + (d_{n-1} - k_s)^2}} \tag{11.4.22}$$

The matrix $\mathbf{P}_{n-1}^{(s)} \cdot \mathbf{A}_s \cdot \mathbf{P}_{n-1}^{(s)T}$ is tridiagonal with two extra elements:

$$\begin{bmatrix} \cdots & & & & & \\ & \times & \times & \times & & \\ & & \times & \times & \times & \mathbf{x} \\ & & & \times & \times & \times \\ & & & \mathbf{x} & \times & \times \end{bmatrix} \tag{11.4.23}$$

We must now reduce this to tridiagonal form with an orthogonal matrix whose last row is $[0, 0, \dots, 0, 1]$ so that the last row of $\bar{\mathbf{Q}}_s^T$ will stay equal to $\mathbf{P}_{n-1}^{(s)}$. This can be done by a sequence of Householder or Givens transformations. For the special form of the matrix (11.4.23), Givens is better. We rotate in the plane $(n-3, n-2)$ to annihilate the $(n-3, n-1)$ element. [By symmetry, the $(n-1, n-3)$ element will also be zeroed.] This leaves us with a tridiagonal form except for the extra elements $(n-4, n-2)$ and $(n-2, n-4)$. We annihilate these with a rotation in the $(n-4, n-3)$-plane, and so on. Thus a sequence of $n-2$ Givens rotations is required. The result is that

$$\mathbf{Q}_s^T = \bar{\mathbf{Q}}_s^T = \bar{\mathbf{P}}_1^{(s)} \cdot \bar{\mathbf{P}}_2^{(s)} \cdots \bar{\mathbf{P}}_{n-2}^{(s)} \cdot \mathbf{P}_{n-1}^{(s)} \tag{11.4.24}$$

where the $\bar{\mathbf{P}}$'s are the Givens rotations and $\mathbf{P}_{n-1}$ is the same plane rotation as in the original algorithm. Then equation (11.4.21) gives the next iterate of $\mathbf{A}$. Note that the shift k_s enters implicitly through the parameters (11.4.22).

The following routine tqli ("Tridiagonal QL Implicit"), based algorithmically on the implementations in [2,3], works extremely well in practice. The number of iterations for the first few eigenvalues might be four or five, say, but meanwhile the off-diagonal elements in the lower right-hand corner have been reduced too. The later eigenvalues are liberated with very little work. The average number of iterations per eigenvalue is typically 1.3–1.6. The operation count per iteration is $O(n)$, with a fairly large effective coefficient, say $\sim 20n$. The total operation count for the diagonalization is then very roughly $\sim 20n \times (1.3–1.6)n \sim 30n^2$. If the eigenvectors are required, the statements indicated by comments are included and there is an additional, much larger, workload of about $6n^3$ operations.

eigen_sym.h

```
void Symmeig::tqli()
```

QL algorithm with implicit shifts to determine the eigenvalues and (optionally) the eigenvectors of a real, symmetric, tridiagonal matrix, or of a real symmetric matrix previously reduced by tred2 (§11.3). On input, d[0..n-1] contains the diagonal elements of the tridiagonal matrix. On output, it returns the eigenvalues. The vector e[0..n-1] inputs the subdiagonal elements of the tridiagonal matrix, with e[0] arbitrary. On output e is destroyed. If the eigenvectors of a tridiagonal matrix are desired, the matrix z[0..n-1][0..n-1] is input as the identity matrix. If the eigenvectors of a matrix that has been reduced by tred2 are required, then z is input as the matrix output by tred2. In either case, column k of z returns the normalized eigenvector corresponding to d[k].

```
{
    Int m,l,iter,i,k;
    Doub s,r,p,g,f,dd,c,b;
    const Doub EPS=numeric_limits<Doub>::epsilon();
    for (i=1;i<n;i++) e[i-1]=e[i];                    Convenient to renumber the el-
    e[n-1]=0.0;                                            ements of e.
    for (l=0;l<n;l++) {
        iter=0;
        do {
            for (m=l;m<n-1;m++) {                     Look for a single small subdi-
                dd=abs(d[m])+abs(d[m+1]);                  agonal element to split the
                if (abs(e[m]) <= EPS*dd) break;            matrix.
            }
            if (m != l) {
                if (iter++ == 30) throw("Too many iterations in tqli");
                g=(d[l+1]-d[l])/(2.0*e[l]);           Form shift.
                r=pythag(g,1.0);
                g=d[m]-d[l]+e[l]/(g+SIGN(r,g));       This is d_m - k_s.
                s=c=1.0;
                p=0.0;
                for (i=m-1;i>=l;i--) {                A plane rotation as in the origi-
                    f=s*e[i];                              nal QL, followed by Givens
                    b=c*e[i];                              rotations to restore tridiag-
                    e[i+1]=(r=pythag(f,g));                onal form.
                    if (r == 0.0) {                   Recover from underflow.
                        d[i+1] -= p;
                        e[m]=0.0;
                        break;
                    }
                    s=f/r;
                    c=g/r;
                    g=d[i+1]-p;
                    r=(d[i]-g)*s+2.0*c*b;
                    d[i+1]=g+(p=s*r);
                    g=c*r-b;
                    if (yesvecs) {
                        for (k=0;k<n;k++) {           Form eigenvectors.
                            f=z[k][i+1];
                            z[k][i+1]=s*z[k][i]+c*f;
```

```
                        z[k][i]=c*z[k][i]-s*f;
                    }
                }
            }
            if (r == 0.0 && i >= l) continue;
            d[l] -= p;
            e[l]=g;
            e[m]=0.0;
        }
    } while (m != l);
  }
}

Doub Symmeig::pythag(const Doub a, const Doub b) {
Computes (a^2 + b^2)^{1/2} without destructive underflow or overflow.
    Doub absa=abs(a), absb=abs(b);
    return (absa > absb ? absa*sqrt(1.0+SQR(absb/absa)) :
        (absb == 0.0 ? 0.0 : absb*sqrt(1.0+SQR(absa/absb))));
}
```

11.4.4 Newer Methods

There are two newer algorithms for tridiagonal symmetric systems that are generally more efficient than the QL method, especially for large matrices. The first is the *divide-and-conquer method* [5]. This method divides the tridiagonal matrix into two halves, solves the eigenproblems in each of the two halves, and then stitches the two solutions together to generate the solution of the original problem. The method is applied recursively, with the QL method used once the matrices are sufficiently small. The method is implemented in LAPACK as `dstevd` and is about 2.5 times faster than the QL method for large matrices.

The fastest method of all for the vast majority of matrices is the *MRRR algorithm* (Multiple Relatively Robust Representations) [6]. As we will see in §11.8, inverse iteration can determine the eigenvectors of a tridiagonal matrix in $O(n^2)$ operations. However, clustered eigenvalues lead to eigenvectors that are not properly orthogonal to one another. Using a procedure like Gram-Schmidt to orthogonalize the vectors is $O(n^3)$. The MRRR algorithm is a sophisticated version of inverse iteration that is $O(n^2)$ without requiring Gram-Schmidt. An implementation is available in LAPACK as `dstegr`.

CITED REFERENCES AND FURTHER READING:

Acton, F.S. 1970, *Numerical Methods That Work*; 1990, corrected edition (Washington, DC: Mathematical Association of America), pp. 331–335.[1]

Wilkinson, J.H., and Reinsch, C. 1971, *Linear Algebra*, vol. II of *Handbook for Automatic Computation* (New York: Springer).[2]

Smith, B.T., et al. 1976, *Matrix Eigensystem Routines — EISPACK Guide*, 2nd ed., vol. 6 of Lecture Notes in Computer Science (New York: Springer).[3]

Stoer, J., and Bulirsch, R. 2002, *Introduction to Numerical Analysis*, 3rd ed. (New York: Springer), §6.6.4.[4]

Cuppen, J.J.M. 1981, "A Divide-and-Conquer Method for the Symmetric Tridiagonal Eigenproblem," *Numerische Mathematik*, vol. 36, pp. 177–195.[5]

Dhillon, I.S., and Parlett, B.N. 2004, "Multiple Representations to Compute Orthogonal Eigenvectors of Symmetric Tridiagonal Matrices," *Linear Algebra and Its Applications*, vol. 387, pp. 1–28.[6]

11.5 Hermitian Matrices

The complex analog of a real symmetric matrix is a Hermitian matrix, satisfying equation (11.0.4). Jacobi transformations can be used to find eigenvalues and eigenvectors, as can Householder reduction to tridiagonal form followed by QL iteration. Complex versions of the previous routines `jacobi`, `tred2`, and `tqli` are quite analogous to their real counterparts. For working routines, consult [1,2].

An alternative, using the routines in this book, is to convert the Hermitian problem to a real symmetric one: If $\mathbf{C} = \mathbf{A} + i\mathbf{B}$ is a Hermitian matrix, then the $n \times n$ complex eigenvalue problem

$$(\mathbf{A} + i\mathbf{B}) \cdot (\mathbf{u} + i\mathbf{v}) = \lambda(\mathbf{u} + i\mathbf{v}) \tag{11.5.1}$$

is equivalent to the $2n \times 2n$ real problem

$$\begin{bmatrix} \mathbf{A} & -\mathbf{B} \\ \mathbf{B} & \mathbf{A} \end{bmatrix} \cdot \begin{bmatrix} \mathbf{u} \\ \mathbf{v} \end{bmatrix} = \lambda \begin{bmatrix} \mathbf{u} \\ \mathbf{v} \end{bmatrix} \tag{11.5.2}$$

Note that the $2n \times 2n$ matrix in (11.5.2) is symmetric: $\mathbf{A}^T = \mathbf{A}$ and $\mathbf{B}^T = -\mathbf{B}$ if $\mathbf{C}$ is Hermitian.

Corresponding to a given eigenvalue λ, the vector

$$\begin{bmatrix} -\mathbf{v} \\ \mathbf{u} \end{bmatrix} \tag{11.5.3}$$

is also an eigenvector, as you can verify by writing out the two matrix equations implied by (11.5.2). Thus, if $\lambda_0, \lambda_1, \ldots, \lambda_{n-1}$ are the eigenvalues of $\mathbf{C}$, then the $2n$ eigenvalues of the augmented problem (11.5.2) are $\lambda_0, \lambda_0, \lambda_1, \lambda_1, \ldots, \lambda_{n-1}, \lambda_{n-1}$; each, in other words, is repeated twice. The eigenvectors are pairs of the form $\mathbf{u} + i\mathbf{v}$ and $i(\mathbf{u} + i\mathbf{v})$; that is, they are the same up to an inessential phase. Thus we solve the augmented problem (11.5.2) and choose one eigenvalue and eigenvector from each pair. These give the eigenvalues and eigenvectors of the original matrix $\mathbf{C}$.

Working with the augmented matrix requires a factor of two more storage than the original complex matrix. In principle, a complex algorithm is also a factor of two more efficient in computer time than is the solution of the augmented problem. In practice, most complex implementations do not achieve this factor unless they are written entirely in real arithmetic. (Good library routines always do this.)

CITED REFERENCES AND FURTHER READING:

Wilkinson, J.H., and Reinsch, C. 1971, *Linear Algebra*, vol. II of *Handbook for Automatic Computation* (New York: Springer).[1]

Smith, B.T., et al. 1976, *Matrix Eigensystem Routines — EISPACK Guide*, 2nd ed., vol. 6 of Lecture Notes in Computer Science (New York: Springer).[2]

11.6 Real Nonsymmetric Matrices

The algorithms for symmetric matrices given in the preceding sections are highly satisfactory in practice. By contrast, it is impossible to design equally satisfactory algorithms for the nonsymmetric case. There are two reasons for this. First, the eigenvalues of a nonsymmetric matrix can be very sensitive to small changes in the matrix elements. Second, the matrix itself can be defective, so that there is no complete set of eigenvectors. We emphasize that these difficulties are intrinsic properties of certain nonsymmetric matrices, and no numerical procedure can "cure" them. The best we can hope for are procedures that don't exacerbate such problems.

The presence of rounding error can only make the situation worse. With finite-precision arithmetic, one cannot even design a foolproof algorithm to determine whether a given matrix is defective or not. Thus current algorithms generally *try* to find a *complete* set of eigenvectors and rely on the user to inspect the results. If any eigenvectors are almost parallel, the matrix is probably defective.

The strategy for finding the eigensystem of a general matrix parallels that of the symmetric case. First we reduce the matrix to a simpler form, and then we perform an iterative procedure on the simplified matrix. The simpler structure we use here is called *Hessenberg* form, defined later in this section. The user interface to the routine is very simple. The declaration

```
Unsymmeig h(a);
```

computes all eigenvalues and eigenvectors of the matrix `a`. The eigenvalues are stored in the complex vector `h.wri` and the corresponding eigenvectors in the columns of the matrix `h.zz`. If `h.wri[i]` is real, the real eigenvector is in `h.zz[0..n-1][i]`. For complex eigenvalues, if `h.wri[i]` has a positive imaginary part, then the complex-conjugate eigenvalue is in `h.wri[i+1]`. Only the eigenvector corresponding to `h.wri[i]` is returned, with the real part in `h.zz[0..n-1][i]` and the imaginary part in `h.zz[0..n-1][i+1]`. The eigenvector corresponding to `h.wri[i+1]` is simply the complex conjugate of this one.

Optional arguments allow you to compute only the eigenvalues, or to input a matrix already in Hessenberg form:

```
Unsymmeig h(a,false);           Only eigenvalues computed.
Unsymmeig h(a,true,true);       Both eigenvalues and eigenvectors, Hessenberg matrix.
```

Here is the routine. The implementations of the various components are discussed in the rest of this section and the next.

eigen_unsym.h

```
struct Unsymmeig {
Computes all eigenvalues and eigenvectors of a real nonsymmetric matrix by reduction to Hessenberg form followed by QR iteration.
    Int n;
    MatDoub a,zz;
    VecComplex wri;
    VecDoub scale;                  Stores scaling from balance.
    VecInt perm;                    Stores permutation from elmhes.
    Bool yesvecs,hessen;

    Unsymmeig(MatDoub_I &aa, Bool yesvec=true, Bool hessenb=false) :
        n(aa.nrows()), a(aa), zz(n,n,0.0), wri(n), scale(n,1.0), perm(n),
        yesvecs(yesvec), hessen(hessenb)
    Computes all eigenvalues and (optionally) eigenvectors of a real nonsymmetric matrix
    a[0..n-1][0..n-1] by reduction to Hessenberg form followed by QR iteration. If yesvecs
    is input as true (the default), then the eigenvectors are computed. Otherwise, only the
    eigenvalues are computed. If hessen is input as false (the default), the matrix is first
    reduced to Hessenberg form. Otherwise it is assumed that the matrix is already in Hessen-
    berg from. On output, wri[0..n-1] contains the eigenvalues of a sorted into descending
    order, while zz[0..n-1][0..n-1] is a matrix whose columns contain the corresponding
```

```
    eigenvectors. For a complex eigenvalue, only the eigenvector corresponding to the eigen-
    value with a positive imaginary part is stored, with the real part in zz[0..n-1][i] and the
    imaginary part in h.zz[0..n-1][i+1]. The eigenvectors are not normalized.
    {
        balance();
        if (!hessen) elmhes();
        if (yesvecs) {
            for (Int i=0;i<n;i++)               Initialize to unit matrix.
                zz[i][i]=1.0;
            if (!hessen) eltran();
            hqr2();
            balbak();
            sortvecs();
        } else {
            hqr();
            sort();
        }
    }
    void balance();
    void elmhes();
    void eltran();
    void hqr();
    void hqr2();
    void balbak();
    void sort();
    void sortvecs();
};
```

11.6.1 Balancing

The sensitivity of eigenvalues to rounding errors during the execution of some algorithms can be reduced by the procedure of *balancing*. The errors in the eigensystem found by a numerical procedure are generally proportional to the Euclidean norm of the matrix, that is, to the square root of the sum of the squares of the elements. The idea of balancing is to use similarity transformations to make corresponding rows and columns of the matrix have comparable norms, thus reducing the overall norm of the matrix while leaving the eigenvalues unchanged. A symmetric matrix is already balanced.

Balancing is a procedure with of order N^2 operations. Thus, the time taken by the procedure `balance`, given below, should never be significant compared to the total time required to find the eigenvalues. It is therefore recommended that you *always* balance nonsymmetric matrices. It never hurts, and it can substantially improve the accuracy of the eigenvalues computed for a badly balanced matrix.

The actual algorithm used is due to Osborne, as discussed in [1]. It consists of a sequence of similarity transformations by diagonal matrices $\mathbf{D}$. To avoid introducing rounding errors during the balancing process, the elements of $\mathbf{D}$ are restricted to be exact powers of the radix base employed for floating-point arithmetic (i.e., 2 for all modern machines, but 16 for some historical mainframe architectures). The output is a matrix that is balanced in the norm given by summing the absolute magnitudes of the matrix elements. This is more efficient than using the Euclidean norm, and equally effective: A large reduction in one norm implies a large reduction in the other.

Note that if the off-diagonal elements of any row or column of a matrix are all zero, then the diagonal element is an eigenvalue. If the eigenvalue happens to be ill-conditioned (sensitive to small changes in the matrix elements), it will have